Tejas Detection + 5 Keypoints (YOLO-Pose)

Study notes from our discussion. Beginner level, explained step by step.

1. Your task in one paragraph

Take a photo that contains a Tejas aircraft and make the computer answer two questions:

1. **Where is the aircraft?** It draws a box around it. This is *object detection*.
2. **Where are its key parts?** It marks 5 points: nose, left wing, right wing, tail, mid. This is *keypoint detection*. Detection and keypoints together are called *pose estimation*.

The team's rule: **the same physical part always gets the same number, from any angle or rotation.** If the left tail is point X in one image, it is X in every image.

For now, the goal is to show on **internet photos** that a neural network can do this, using your Ubuntu PC with an RTX A4000, and then show the results to your team. The 1 CPU core / 30 fps speed target is postponed until later, after the team sees that the method works.

2. Why keypoints, not just a box?

A box only says "an aircraft is here". Five points also tell you which way it faces, how it is tilted, and where its centre is. The nose-to-tail line gives the heading. The wing-to-wing line gives the tilt (roll). The mid point gives the position.

3. The 5 keypoints (my proposal, confirm with your team)

#	Point	Meaning
0	nose	front tip, gives the head direction
1	left wingtip	the aircraft's OWN left (the pilot's left), not the left of the image
2	right wingtip	the aircraft's OWN right
3	tail	rear end
4	mid	centre of the body, gives the exact position

> **Rules:** never change the order after you start labelling. Same point = same number in every image. When the aircraft flies toward the camera, its left wing appears on the RIGHT of the picture, and that is still point 1. If a point is hidden, mark it "not visible" instead of guessing wildly.

Honest risk: when the aircraft is far away, tiny, or exactly head-on, even a human cannot easily tell left from right. Check those images separately in your results.

4. Key concepts

Image = numbers

A photo is a grid of pixel values. The model reads the numbers and predicts other numbers: box coordinates and point coordinates.

CNN (convolutional neural network)

A network that learns patterns: first edges, then shapes such as a wing or nose, then whole objects. You do not write the rules. You show examples and it learns.

YOLO and YOLO-Pose

YOLO is a fast detector that looks at the image once. YOLO-Pose also predicts the keypoints in the same pass. One detection looks like: `box + class + 5 x (x, y, visibility)`.

Training and loss

The model guesses, measures its error (the *loss*), adjusts its internal numbers, and repeats thousands of times. The loss has a box part, a class part and a keypoint part.

Transfer learning (fine-tuning)

You start from a model that already learned to find keypoints on humans and adapt it to aircraft. This is why about 100 to 300 images can be enough for a first result.

Train / validation / test

Train images teach the model. Validation images check it during training. Test images are the final exam and are never used for training.

Augmentation

Automatic random changes to training images (rotate, zoom, mirror, change brightness) to add variety. Mirroring swaps left and right, so `flip_idx` in `data.yaml` tells the model that point 1 becomes point 2 after a flip.

Overfitting

The model memorizes the training photos and fails on new ones. The warning sign is a great training score with poor results on new photos.

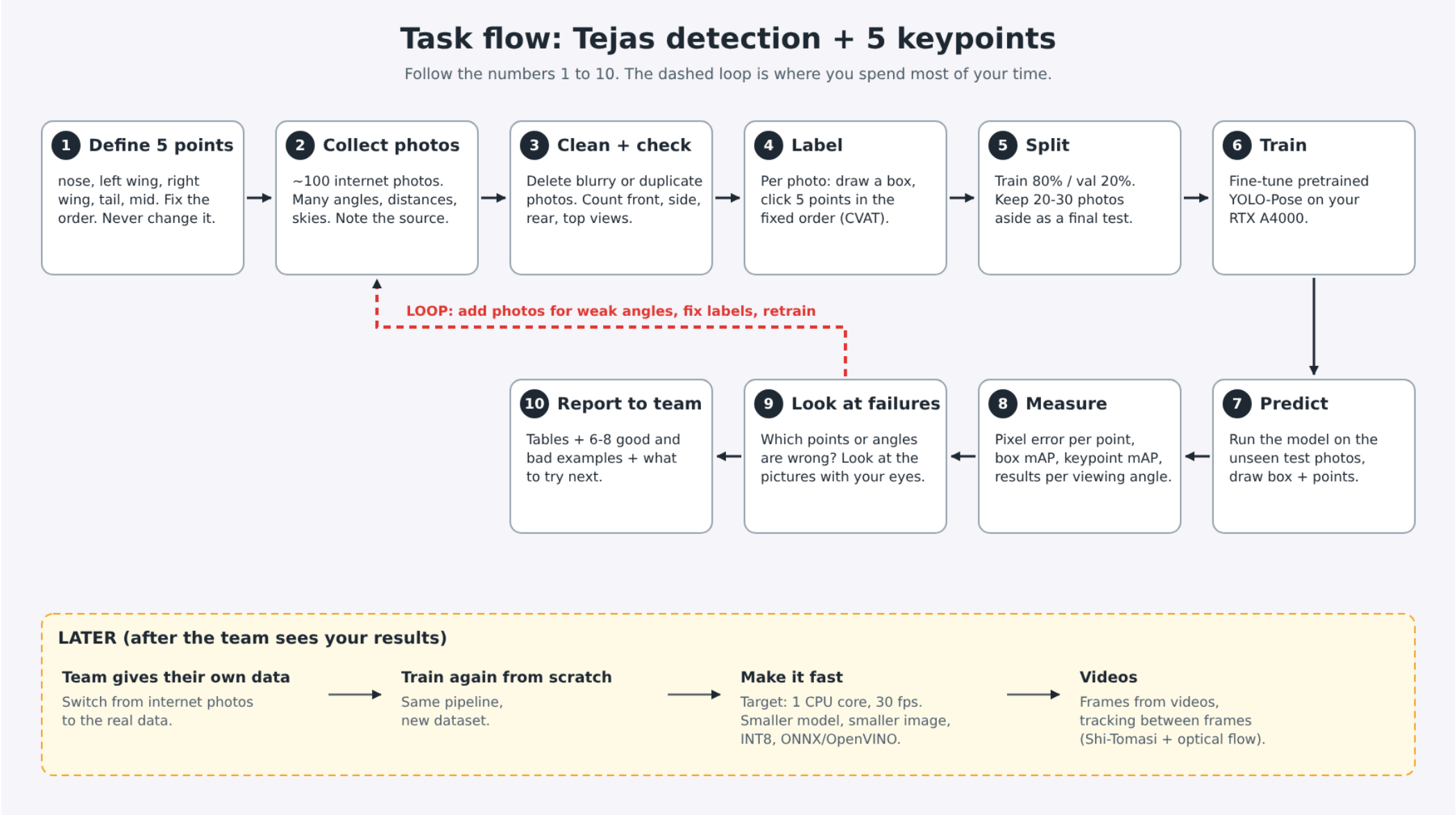
5. Algorithms in your lecture notes

Algorithm	What it does	Role in your work
CNN / YOLO-Pose	Learns boxes and named keypoints	**Main method**
Viola-Jones	Old, very fast detector for fixed views (for example frontal faces)	Weak when the object rotates, so use only as a baseline
Harris, Shi-Tomasi	Find sharp corners, but cannot name them	Cannot say "this is the nose". Later useful for tracking points between video frames
Heatmap + offset head (from your notes)	Predicts a probability map for each point, plus a small offset	Usually more accurate but heavier
Direct regression (YOLO-Pose)	Predicts the coordinates directly	Simpler and faster, the right starting point

Their earlier CNN worked but was slow. The speed fix comes later: a smaller model, a smaller image size, INT8 quantization, and tracking points between frames.

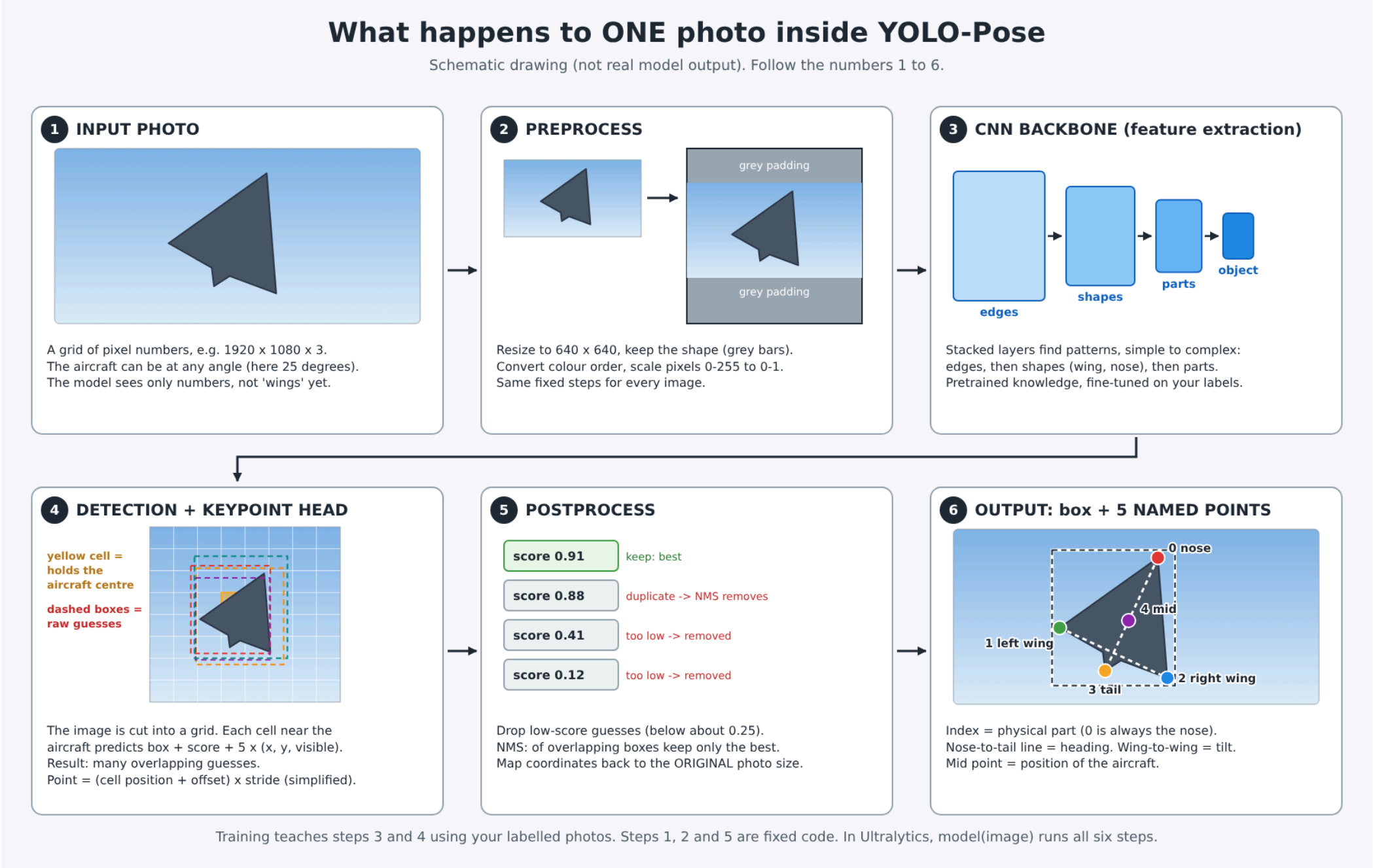
6. Diagram: your task flow

Follow the numbers 1 to 10. The red dashed loop is where you spend most of your time.



7. Diagram: what happens to ONE photo inside YOLO-Pose

This is a schematic drawing, not real model output. Real results will come from your trained model.



8. Preprocessing, inference, postprocessing

These are the three stages that run every time the model processes one image. The time budget of 33 ms per frame (30 fps) includes all three.

Stage	What happens
Preprocessing	Resize to the model input size (for example 640 x 640) with padding, convert colour order (BGR to RGB), scale pixels from 0-255 to 0-1, arrange into the shape the network expects.
Inference	The image goes through the network once. It outputs many candidate boxes, each with a score and 5 predicted points. Each cell of the grid near the aircraft predicts a box and the points as offsets from the cell, so a point is roughly (cell position + offset) x stride. Usually the most expensive stage.
Postprocessing	Drop low-score candidates, apply **NMS (non-maximum suppression)** to keep only the best box per aircraft, convert coordinates back to the ORIGINAL image size, draw the box and points.

`photo -> PREPROCESS -> INFERENCE -> POSTPROCESS -> box + 5 points`

Ultralytics does all three for you when you call `model(image)`. Do not confuse preprocessing (every time) with augmentation (only during training). A common beginner bug is forgetting to map points back to the original image size.

9. What is labelling?

Labelling (annotation) means **you teach the computer the correct answer by marking it on each photo**. For every image: draw a box around the aircraft, then click the 5 keypoints in the fixed order. The tool saves a text file with the same name as the image (`tejas_001.jpg` and `tejas_001.txt`).

Each line of the label file has the format (all values scaled 0 to 1, `v` = visibility):

```
class cx cy w h  kx1 ky1 v1  kx2 ky2 v2  kx3 ky3 v3  kx4 ky4 v4  kx5 ky5 v5
```

The model learns only from these marks. Wrong or inconsistent labels mean the model learns wrong things, so labelling is the most important and slowest part. Tools: CVAT, Label Studio or Roboflow. Export in the YOLO keypoints format.

10. What to do after collecting photos

1. **Clean:** delete blurry, tiny, duplicate or wrong-aircraft photos. Keep a sheet with source link and licence for each photo.
2. **Check variety:** count photos for side, front, rear, top and far-away views. Collect more where there are very few.
3. **Rename** files simply (`img_001.jpg`).
4. **Label** about 100 images (class `tejas`, 5 keypoints in the agreed order).
5. **Check labels** by viewing about 20 labelled images with the points drawn.

6. **Export** in YOLO keypoints format.
7. **Split** into train and validation, and keep 20-30 photos aside as a final test.
8. **Train**, then **predict**, then **look** at the failures, then improve.

> **Time-saver (pre-labelling):** label about 40 images, train a quick model, let it predict on the rest, then only correct its mistakes. Fixing is much faster than clicking from scratch.

> **Beware near-duplicates:** many web photos come from the same airshow or event. If near-identical photos land in both train and test, the score looks better than reality. Keep photos from the same source on the same side of the split and test on photos from other sources. For video frames later, split by video, not by frame.

11. Step-by-step plan with commands

Setup (once, on your Ubuntu PC)

```
nvidia-smi
python3 -m venv venv && source venv/bin/activate
pip install ultralytics
python -c "import torch; print(torch.cuda.is_available())"   # should print True
```

Quick test with a pretrained model

```
from ultralytics import YOLO
model = YOLO("yolo11n-pose.pt")     # exact file names depend on your Ultralytics
version
results = model("image.jpg")
for r in results:
    print(r.boxes.xyxy)             # boxes
    print(r.keypoints.xy)           # keypoint coordinates
    print(r.keypoints.conf)         # keypoint confidences
```

Project files (in tejas_pose.zip)

File	Purpose
data.yaml	5 keypoints, `kpt_shape: [5, 3]`, `flip_idx: [0, 2, 1, 3, 4]`
extract_frames.py	Videos to frames (for later)
make_split.py	Train/val split. Default: each photo on its own. `--mode video` for video frames
train.py	Fine-tunes a pretrained pose model on the A4000
predict.py	Draws box + 5 points on test images and prints pixel error

Order of commands

```
python make_split.py labeled/images labeled/labels
python train.py --fast-test        # 3 epochs, only checks nothing crashes
python train.py                    # real training
python predict.py test_images/     # results in runs/predict/
```

If you get "CUDA out of memory", use `python train.py --batch 8`.

12. How to measure success

- **Pixel error:** distance between the predicted point and your labelled point. Easiest to explain to your team.
- **Box mAP:** how well the boxes match.
- **OKS and keypoint mAP:** OKS (Object Keypoint Similarity) is the pose version of box overlap (IoU). Keypoint mAP is the standard score built on it.
- **Per-angle results:** accuracy for side, front, rear and top views separately.

A good first result is moderate accuracy, honestly explained, for example: "nose and wingtips are accurate to about X pixels on side views, but left and right wings get confused head-on." That tells the team which data to supply next.

13. Common beginner mistakes

- Labelling inconsistently (the same point given different numbers).
- Training before checking the labels by eye.
- Too little variety (all side views).
- Near-duplicate photos in both train and test.
- Forgetting to map predicted points back to the original image size.

14. Later: team data, speed (1 CPU core, 30 fps), videos

- When the team gives their own data, repeat the same pipeline from scratch on it.
- At 30 fps you have about 33 ms per frame on one core, including all three stages. Ways to get there: the smallest model (nano), a smaller input size (320 or less), fewer keypoints, export to ONNX Runtime / OpenVINO / NCNN, and INT8 quantization.
- Measure early on one core: `taskset -c 0 python benchmark.py` with threads set to 1. Ask the team for the exact target CPU, resolution and what "30 fps" includes.
- For video: run the CNN every few frames and track the points in between with Shi-Tomasi corners plus Lucas-Kanade optical flow, which is very cheap.

15. Questions to ask your team, and open items

1. Are the 5 points (nose, left wing, right wing, tail, mid) acceptable? Their words: head, tail and mid points to define the exact position, consistent from any angle.

2. Heatmaps + offsets or direct regression for the keypoint head? If they do not know, start with regression.
3. What is the target CPU, and what image resolution?
4. May you use data from the team later, and are there any data-sharing or confidentiality limits?
5. Some words in your handwritten notes were hard to read (the word under "5-point" and the line starting "(*) to check high impact..."). Confirm their meaning with your team.

16. Glossary

Bounding box	Rectangle around the object.
Keypoint	A named point on the object (nose, wingtip...).
Pose estimation	Finding the object's keypoints to describe its position and orientation.
Confidence / score	How sure the model is about a detection or point.
NMS	Removes duplicate boxes, keeping the best.
mAP	Mean average precision, the standard detection score.
OKS	Object Keypoint Similarity, the keypoint version of IoU.
Epoch	One full pass over the training images.
Fine-tuning	Adapting a pretrained model to your data.
Quantization (INT8)	Storing the model in smaller numbers for faster CPU speed, with a small accuracy loss.
ONNX / OpenVINO	Formats and runtimes for running models fast on CPU.